



Nombre y Código del Estudiante

José Rafael Pérez García
22611142

Ian Mauricio Roberto Zapata Morales
22611429

Catedrático

Ing. Nazarena Idiaquez

Asignatura y sección

Laboratorio de Programación
Sección 1037

Actividad Asignada

Proyecto Final de Laboratorio: Juego de Búsqueda de Colores

Lugar y Fecha

El Progreso, Yoro
22 de junio de 2026

Índice

Índice	2
Objetivos	4
Objetivo general	4
Objetivos específicos	4
Investigaciones	5
Temporización visual sin bloqueo de hilos: Timeline y KeyFrame	5
Comunicación cliente-servidor mediante sockets TCP	5
Integración de interfaces gráficas: JavaFX, FXML y Scene Builder	5
Requisitos para ejecutar el proyecto	7
Java Development Kit (JDK) - Versión 11 o superior	7
JavaFX SDK 13	7
Apache Maven	7
Entorno de Desarrollo Integrado (IDE)	7
Conexión de red local (modo multijugador)	7
Diccionario de variables	8
Clase App (en App.java)	8
Clase PrincipalController (en PrincipalController.java)	8
Clase MultijugadorController (en MultijugadorController.java)	8
Clase ConexionMenuController (en ConexionMenuController.java)	9
Clase Conexion (en Conexion.java)	9
Clase Colores (en colores.java)	10
Clase Puntaje (en puntaje.java)	10
Ejemplos	11
Descripción del Juego y Reglas	13
Arquitectura del Software y Componentes Clave	14
Separación entre vista y lógica mediante FXML	14
Patrón Singleton (clase Conexion)	14
Arquitectura cliente-servidor y concurrencia mediante hilos	14
Comunicación desacoplada mediante callbacks	14
Modelo de datos del juego (Colores y Puntaje)	14
Ventanas emergentes de resultado (clase ventana)	15
Conclusiones	16

Introducción

El presente proyecto consiste en el diseño y desarrollo de un juego de búsqueda de colores, basado en el efecto Stroop, creado en el lenguaje de programación Java y ejecutado mediante una interfaz gráfica construida con JavaFX. El programa fue desarrollado bajo el paradigma de Programación Orientada a Objetos (POO), utilizando archivos FXML para la separación entre la vista y la lógica de control, temporizadores integrados de JavaFX (Timeline y KeyFrame) y comunicación en red mediante sockets TCP para habilitar un modo multijugador entre dos computadoras.

La realización de este proyecto implicó la investigación de herramientas gráficas y de concurrencia propias de Java, dado que el juego, además de poner a prueba la atención y la velocidad de respuesta del jugador frente a estímulos visuales contradictorios, ofrece dos experiencias de juego distintas: una modalidad individual contra el tiempo y una modalidad competitiva en red entre dos jugadores conectados a través de la misma conexión local.

Objetivos

Objetivo general

- Desarrollar un videojuego funcional de búsqueda de colores basado en el efecto Stroop, utilizando JavaFX para la construcción de la interfaz gráfica y sockets TCP para la comunicación en red, aplicando los principios de la Programación Orientada a Objetos.

Objetivos específicos

- Implementar una interfaz gráfica interactiva mediante JavaFX y archivos FXML, separando la lógica del juego de su representación visual.
- Aplicar temporizadores basados en el componente Timeline de JavaFX para controlar la duración de cada partida sin bloquear el hilo principal de la interfaz.
- Desarrollar un módulo de comunicación en red mediante sockets TCP que permita disputar una partida multijugador entre dos computadoras distintas.
- Aplicar el paradigma POO mediante clases independientes (Colores, Puntaje, Conexion, ventana) para el manejo y almacenamiento de los datos empleados por el juego, favoreciendo la reutilización de código.

Investigaciones

Para la concepción y desarrollo del software, se investigaron e implementaron distintas herramientas y conceptos, entre los cuales se encuentran:

Temporización visual sin bloqueo de hilos: Timeline y KeyFrame

A diferencia de una aplicación de consola, donde el hilo principal puede detenerse de forma síncrona en espera de una entrada de teclado, una interfaz gráfica no puede congelarse sin afectar la experiencia del usuario. Por ello, se investigó y utilizó el componente Timeline de JavaFX en conjunto con KeyFrame y Duration, los cuales permiten ejecutar una acción de forma recurrente (en este caso, descontar un segundo del cronómetro) sin bloquear el hilo de la interfaz (JavaFX Application Thread). Esta alternativa evita la complejidad de sincronizar manualmente hilos mediante Thread.sleep() o temporizadores de bajo nivel, delegando dicho control al propio framework gráfico.

Comunicación cliente-servidor mediante sockets TCP

Para el modo multijugador en red se investigaron las clases Socket y ServerSocket del paquete java.net, las cuales permiten establecer una comunicación bidireccional entre dos computadoras mediante el protocolo TCP. Dado que la lectura de datos de un socket (BufferedReader.readLine()) es una operación bloqueante, fue necesario ejecutar dicha escucha en un hilo (Thread) secundario, evitando que la interfaz gráfica del jugador se congele mientras se espera un mensaje del oponente. Adicionalmente, se investigó el uso de referencias funcionales del tipo Consumer<T>, las cuales permiten notificar al controlador de la interfaz cuando llega un nuevo puntaje o cuando se pierde la conexión, sin acoplar directamente la lógica de red con la lógica visual.

Integración de interfaces gráficas: JavaFX, FXML y Scene Builder

Se investigó el ecosistema de JavaFX en conjunto con FXML como la solución idónea para construir la interfaz gráfica del juego bajo un enfoque moderno y desacoplado. JavaFX actúa como el framework de renderizado y gestor del ciclo de vida de la aplicación, mientras que FXML, al ser un lenguaje declarativo basado en XML, permite definir la estructura de la interfaz de manera totalmente independiente de la lógica del negocio. Esta separación estricta entre la presentación visual y el código de control garantiza una arquitectura de software limpia que sigue de cerca el patrón de diseño de software de tipo Modelo-Vista-Controlador (MVC).

Para agilizar y optimizar este proceso de maquetación, se incorporó la herramienta visual interactiva Scene Builder de Gluon, la cual genera automáticamente el archivo de marcas XML estructurado. Esta herramienta facilita enormemente el diseño y la distribución de los componentes en pantalla, permitiendo enlazar directamente la escena física con su clase controladora en Java utilizando la propiedad fx:controller.

Asimismo, provee un mapeo rápido y limpio de eventos de acción (como clics en botones) y propiedades de componentes mediante el uso de identificadores únicos (fx:id) que se inyectan en el controlador.

Requisitos para ejecutar el proyecto

Para garantizar la correcta compilación y ejecución del software, se requieren los siguientes componentes tecnológicos debidamente configurados:

Java Development Kit (JDK) - Versión 11 o superior

Java es un lenguaje que se compila a un código intermedio conocido como bytecode, el cual solo puede ser interpretado y ejecutado por la Máquina Virtual de Java (JVM). El JDK proporciona las bibliotecas estándar necesarias, el compilador (javac) y la propia JVM. El proyecto fue configurado para compilarse con la versión 11 de Java, según se especifica en el archivo de gestión de dependencias pom.xml.

JavaFX SDK 13

JavaFX es la biblioteca encargada de construir la interfaz gráfica del juego. El proyecto depende específicamente de los módulos javafx-controls (botones, etiquetas y demás componentes visuales), javafx-fxml (carga de las vistas definidas en archivos .fxml) y javafx-media (reproducción de recursos de audio).

Apache Maven

Maven es la herramienta utilizada para gestionar las dependencias del proyecto y automatizar su compilación y ejecución. Mediante el complemento javafx-maven-plugin, configurado en el pom.xml, es posible ejecutar la aplicación directamente con el comando `mvn clean javafx:run`, sin necesidad de configurar manualmente las rutas de los módulos de JavaFX.

Entorno de Desarrollo Integrado (IDE)

El proyecto fue desarrollado utilizando Apache NetBeans, aunque su estructura basada en Maven permite abrirlo y ejecutarlo en cualquier entorno de desarrollo compatible (IntelliJ IDEA, Visual Studio Code con la extensión de Java, entre otros), siempre que cuente con soporte para proyectos Maven y JavaFX.

Conexión de red local (modo multijugador)

Para disputar una partida en modo multijugador, ambas computadoras deben encontrarse dentro de la misma red local (LAN o Wi-Fi) y tener disponible el puerto 12345, el cual es utilizado por la aplicación para establecer la conexión entre el servidor y el cliente.

Diccionario de variables

A continuación, se catalogan y describen los atributos más relevantes que gobiernan la lógica del sistema, organizados por clase. Se distingue entre los elementos de la interfaz gráfica, marcados con la anotación `@FXML` (inyectados automáticamente por el `FXMLLoader` desde su archivo `.fxml` correspondiente), y las variables y atributos internos declarados directamente en el código Java de cada clase.

Clase App (en `App.java`)

Nombre de variable/atributo	Tipo de dato	Descripción
scene	Scene (static)	Escena principal de la aplicación, compartida estáticamente entre todas las vistas para poder cambiar de pantalla (<code>setRoot</code>) sin crear una nueva ventana.

Clase `PrincipalController` (en `PrincipalController.java`)

Variables y atributos internos de la clase:

Nombre de variable/atributo	Tipo de dato	Descripción
colores	Colores	Instancia utilizada para generar de forma aleatoria la palabra y el color visual de cada ronda.
puntos	Puntaje	Instancia que almacena y gestiona el marcador del jugador durante la partida.
venta	ventana	Instancia utilizada para mostrar la ventana emergente con el resultado final de la partida.
tiempo	int	Segundos restantes de la cuenta regresiva; inicia en 30 y disminuye cada segundo.
cronometro	Timeline	Temporizador de JavaFX que descuenta un segundo por cada <code>KeyFrame</code> ejecutado, sin bloquear la interfaz.

Clase `MultijugadorController` (en `MultijugadorController.java`)

Variables y atributos internos de la clase:

Nombre de variable/atributo	Tipo de dato	Descripción
conexion	Conexion	Referencia a la instancia Singleton de la conexión de red, compartida con la pantalla de conexión previa.
puntos	Puntaje	Marcador local del jugador en la partida multijugador.
colores	Colores	Instancia utilizada para generar de forma aleatoria la palabra y el color visual de cada ronda.
venta	ventana	Instancia utilizada para mostrar la ventana emergente con el resultado de la partida.
tiempo	int	Segundos restantes de la cuenta regresiva local; inicia en 30.
cronometro	Timeline	Temporizador de JavaFX equivalente al del modo individual.

Clase ConexionMenuController (en ConexionMenuController.java)

Variables y atributos internos de la clase:

Nombre de variable/atributo	Tipo de dato	Descripción
conexion	Conexion	Referencia a la instancia Singleton encargada de gestionar la conexión de red.
PUERTO	int (final)	Puerto fijo (12345) utilizado tanto por el servidor como por el cliente para establecer la conexión.

Clase Conexion (en Conexion.java)

Nombre de variable/atributo	Tipo de dato	Descripción
instance	Conexion (static)	Única instancia de la clase, accesible globalmente mediante el método getInstance().
soc	Socket	Socket utilizado para el envío y recepción de datos una vez establecida la conexión.

Nombre de variable/atributo	Tipo de dato	Descripción
servs	ServerSocket	Socket de servidor utilizado únicamente por el jugador que crea la partida, en espera de un oponente.
op, ip	PrintWriter, BufferedReader	Flujos de salida y entrada de datos utilizados para enviar y leer los puntajes a través de la red.
conectado	boolean	Bandera que indica si la conexión con el oponente se encuentra activa.
PuntoRecibido	Consumer<Integer>	Función de retorno (callback) que se ejecuta cada vez que se recibe un nuevo puntaje del oponente.
alDesconectar	Consumer<Boolean>	Función de retorno (callback) que se ejecuta cuando se detecta la desconexión del oponente.

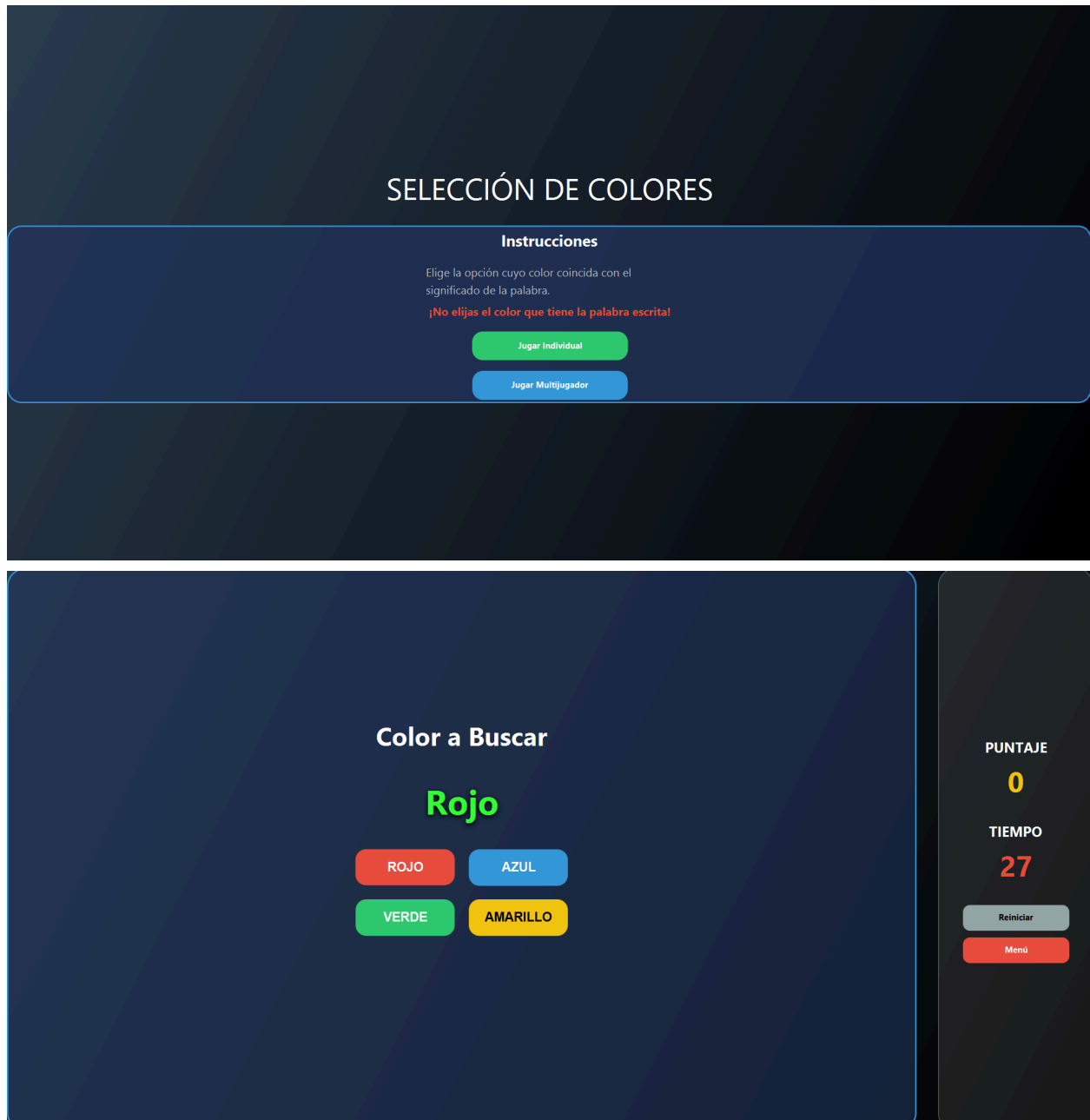
Clase Colores (en colores.java)

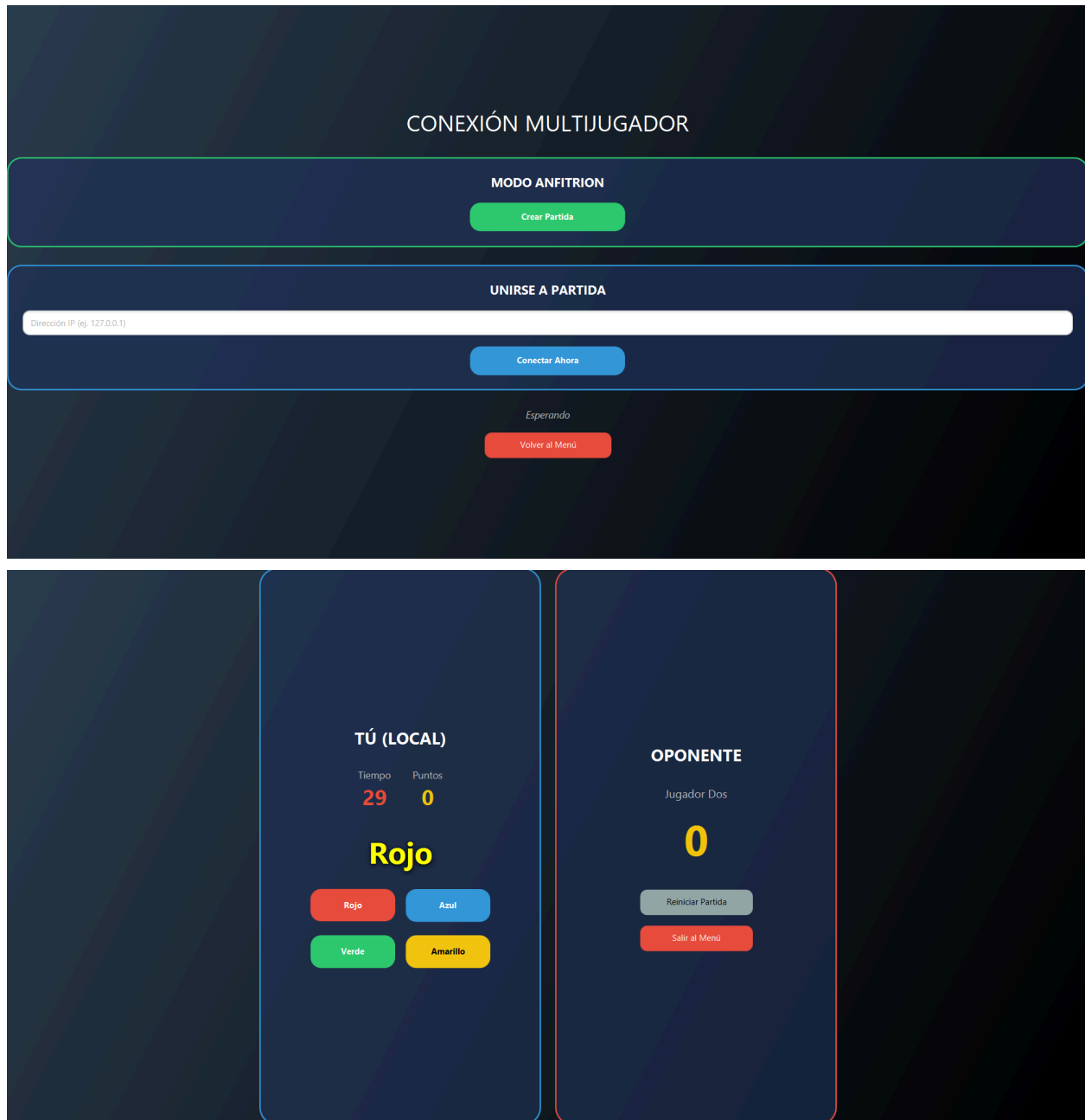
Nombre de variable/atributo	Tipo de dato	Descripción
r	Random	Generador de números aleatorios utilizado para seleccionar la palabra y el color visual de cada ronda.
colores	String[]	Arreglo con los nombres de los colores disponibles: Rojo, Azul, Verde y Amarillo.
numcolor	String[]	Arreglo con los códigos hexadecimales correspondientes a cada color visual.

Clase Puntaje (en puntaje.java)

Nombre de variable/atributo	Tipo de dato	Descripción
valor	int	Marcador del jugador. Se incrementa en 1 con cada acierto y se reduce en 1 con cada error, sin descender de cero.

Ejemplos





Descripción del Juego y Reglas

- El reto: en cada ronda se muestra una palabra que representa un color (Rojo, Azul, Verde o Amarillo), pero pintada con un color visual que puede coincidir o no con su significado. El jugador debe presionar el botón correspondiente al significado de la palabra, sin guiarse por el color con el que está dibujada.
- Los modos de juego: el jugador puede elegir entre una partida individual contra el tiempo, o una partida multijugador en red, donde dos computadoras se conectan mediante una IP y un puerto compartido (servidor y cliente).
- La puntuación: cada respuesta correcta suma 1 punto al marcador; cada respuesta incorrecta resta 1 punto, sin que el marcador pueda descender por debajo de cero.
- El tiempo: cada partida cuenta con una cuenta regresiva de 30 segundos, visible en pantalla, controlada mediante un temporizador de JavaFX (Timeline).
- Condiciones de fin de partida (modo individual): la partida finaliza si el tiempo llega a cero, o de forma anticipada si el marcador desciende a cero tras una respuesta incorrecta.
- Condiciones de fin de partida (modo multijugador): la partida finaliza cuando el tiempo del jugador local llega a cero; el sistema compara entonces el puntaje propio contra el puntaje recibido del oponente a través de la red para determinar si el resultado fue una victoria, una derrota o un empate.
- El reinicio: al finalizar una partida, el jugador puede reiniciarla (botón Reiniciar) sin necesidad de cerrar la aplicación, reestableciendo el marcador y el temporizador.

Arquitectura del Software y Componentes Clave

El código fuente se organiza en módulos lógicos y patrones de diseño acoplados de forma estratégica para garantizar la mantenibilidad y la separación de responsabilidades:

Separación entre vista y lógica mediante FXML

Cada pantalla del juego (menú, conexión multijugador, modo individual y modo multijugador) está definida en un archivo FXML independiente, el cual describe la disposición visual de los componentes. Cada archivo FXML se enlaza a una clase controladora de Java mediante el atributo `fx:controller`, y cada componente visual se conecta a un atributo del controlador mediante `fx:id`. Esta separación permite modificar el diseño visual de una pantalla sin necesidad de alterar su lógica interna, y viceversa.

Patrón Singleton (clase Conexion)

Para evitar crear múltiples instancias de la conexión de red al navegar entre la pantalla de configuración (ConexionMenuController) y la pantalla de juego (MultijugadorController), se implementó el patrón de diseño Singleton en la clase Conexion. El método estático `getInstance()` garantiza que ambos controladores trabajen siempre sobre el mismo objeto de conexión, sin necesidad de pasarlo manualmente entre pantallas.

Arquitectura cliente-servidor y concurrencia mediante hilos

El modo multijugador se apoya en una arquitectura cliente-servidor sencilla: un jugador crea la partida (ServerSocket en espera de conexión) y el otro se une indicando la dirección IP del primero (Socket cliente). Dado que tanto la espera de una conexión entrante como la lectura continua de mensajes son operaciones bloqueantes, ambas se ejecutan dentro de hilos (Thread) secundarios. Las actualizaciones a los componentes visuales que se originan desde estos hilos (por ejemplo, mostrar el puntaje recibido del oponente) se sincronizan de vuelta con el hilo de la interfaz mediante `Platform.runLater()`, evitando errores de concurrencia propios de JavaFX.

Comunicación desacoplada mediante callbacks

En lugar de que la clase Conexion dependa directamente de los controladores de la interfaz, esta expone dos métodos (`ponerPuntos` y `setAIDesconectar`) que reciben referencias funcionales del tipo `Consumer<T>`. De esta forma, cada controlador define qué hacer cuando llega un nuevo puntaje o cuando se pierde la conexión, sin que la clase de red necesite conocer los detalles de la interfaz gráfica.

Modelo de datos del juego (Colores y Puntaje)

Las clases Colores y Puntaje encapsulan, respectivamente, la generación aleatoria de palabras y colores visuales, y el manejo del marcador del jugador. Esta separación permite que tanto el modo individual como el modo multijugador reutilicen exactamente la misma lógica de juego, evitando la duplicación de código entre ambos controladores.

Ventanas emergentes de resultado (clase ventana)

Al finalizar una partida, la clase ventana genera de forma dinámica una ventana modal (Stage) independiente de la ventana principal. Mediante la sobrecarga del método `mensaje()`, esta misma clase es capaz de mostrar un resultado simple en el modo individual, o de comparar dos puntajes (local y remoto) en el modo multijugador para determinar el resultado final de la partida.

Conclusiones

El desarrollo del proyecto permitió afianzar conceptos de programación gráfica en Java mediante JavaFX y FXML, logrando una clara separación entre la presentación visual y la lógica de control del juego.

El uso del patrón de diseño Singleton en la clase Conexion resolvió de forma elegante el problema de compartir una misma conexión de red entre distintas pantallas de la aplicación, sin duplicar instancias ni acoplar fuertemente los controladores.

La implementación del modo multijugador evidenció la importancia de delegar las operaciones bloqueantes de red (esperar conexiones, leer mensajes) a hilos secundarios, sincronizando los resultados con la interfaz gráfica mediante Platform.runLater() para evitar que la aplicación se congele.

El diseño basado en clases independientes (Colores, Puntaje, ventana) demostró que la Programación Orientada a Objetos reduce la redundancia de código al permitir que tanto el modo individual como el multijugador reutilicen la misma lógica de juego.

Como mejora pendiente, el proyecto incluye recursos de audio (efectos de acierto y error) que aún no se encuentran integrados en la lógica final del juego, por lo que su incorporación en una versión futura permitiría enriquecer la retroalimentación sensorial de la experiencia de juego.